

Sprint 17 Structured Notes: CRUD and Client-Side Storage

0. What Sprint 17 Is About

Sprint 17 teaches you how a browser app can remember simple data.

Before Sprint 17, most of your data lived only while the page was open.

Example:

```
let tasks = ["Study JavaScript", "Practice DOM"];
```

This works, but only temporarily.

If the user refreshes the page, the data is lost because the array was stored only in JavaScript memory.

Sprint 17 teaches how to store data in the browser so the app can load it again later.

Main goal:

Create, read, update, delete, and persist simple data in the browser.

Simple meaning:

Sprint 17 teaches you how to build small apps that can remember user data after refresh.

By the end of Sprint 17, you should understand:

- CRUD
- create operations
- read operations
- update operations
- delete operations
- persistence
- in-memory data
- browser storage
- `localStorage`
- `sessionStorage`
- storage keys and values
- why browser storage stores strings
- `JSON.stringify()`

- `JSON.parse()`
 - loading saved data
 - fallback values like `|| []`
 - safe loading with `try...catch`
 - checking stored data with `Array.isArray()`
 - saving after every change
 - removing stored data
 - `removeItem()`
 - `clear()`
 - HTTP method mapping
 - client-side storage safety
 - trust boundaries
 - cookies at a beginner level
 - Cache API at a beginner level
 - IndexedDB at a beginner level
 - service workers at a beginner level
 - common beginner storage mistakes
 - a saved notes or task-list pattern
-

1. Current Progress Before Sprint 17

Sprint 17 comes after you already know the main JavaScript and browser basics.

Sprints 1–5 Review

You learned basic JavaScript:

- values
- variables
- strings
- numbers
- booleans
- conditions
- functions
- scope

These help Sprint 17 because browser storage works with values and functions.

Example:

```
const username = "Sangeeth";
localStorage.setItem("username", username);
```

Here, you store a string value using a variable.

Sprints 6–10 Review

You learned how to organize and process data:

- arrays
- loops
- objects
- callbacks
- array methods
- Sets
- Maps

These help Sprint 17 because stored app data is usually an array of objects.

Example:

```
const tasks = [  
  { text: "Study JavaScript", done: false },  
  { text: "Practice DOM", done: true }  
];
```

This is an array.

Each task is an object.

That kind of structure is common in saved browser apps.

Sprints 11–13 Review

You learned browser interaction:

- browser APIs
- DOM selection
- DOM updates
- events
- timing
- dialogs
- canvas basics

These help Sprint 17 because a real storage app usually responds to user actions.

Example:

```
button.addEventListener("click", () => {  
  addTask("New task");  
});
```

The user clicks a button.

JavaScript updates the data.

Then Sprint 17 teaches you to save that data.

Sprint 14 Review

You learned accessible dynamic JavaScript.

This matters because saved data apps often show messages like:

- Saved successfully.
- Task deleted.
- Notes cleared.

Those messages should be understandable to users and assistive technology.

Example:

```
<p id="status" aria-live="polite"></p>
```

```
status.textContent = "Task saved.";
```

Sprint 15 Review

You learned forms and validation.

This matters because saved data often comes from form input.

Example:

```
<form id="taskForm">  
  <input id="taskInput" required>
```

```
<button type="submit">Add task</button>
</form>
```

```
form.addEventListener("submit", event => {
  event.preventDefault();
  addTask(taskInput.value);
});
```

Sprint 16 Review

You learned dates and time.

This matters because saved records often include timestamps.

Example:

```
const note = {
  text: "Learn localStorage",
  createdAt: new Date().toISOString()
};
```

`createdAt` stores when the note was created.

2. The Main Problem Sprint 17 Solves

Before Sprint 17, your data can disappear when the page reloads.

Example:

```
let notes = [];

notes.push("Learn storage");
console.log(notes);
```

This works while the page is open.

But if the user refreshes, JavaScript starts again from the beginning:

```
let notes = [];
```

So the array becomes empty again.

That is the problem.

Sprint 17 solves it with browser storage.

Simple idea:

```
User creates data
↓
JavaScript stores it in an array
↓
JavaScript saves the array into browser storage
↓
Page reloads
↓
JavaScript loads the saved data back
```

3. In-Memory Data vs Persistent Data

In-Memory Data

In-memory data lives only while the current JavaScript program is running.

Example:

```
let score = 10;
```

The value `10` is in memory.

If the page refreshes, the script starts again, and the old memory is gone.

Simple meaning:

In-memory data is temporary.

Persistent Data

Persistent data stays available after the page reloads.

Example:

```
localStorage.setItem("score", "10");
```

Now the browser stores the score.

Later, you can read it again:

```
const score = localStorage.getItem("score");  
console.log(score);
```

Simple meaning:

Persistent data is remembered after refresh.

4. CRUD

CRUD stands for:

Letter	Meaning	Simple App Example
C	Create	Add a new note
R	Read	Show all notes
U	Update	Edit a note
D	Delete	Remove a note

CRUD is not only a JavaScript idea.

It is a general software idea.

Most apps do CRUD.

Examples:

- A notes app creates, reads, updates, and deletes notes.
- A todo app creates, reads, updates, and deletes tasks.
- A shopping cart creates, reads, updates, and deletes cart items.

- A contacts app creates, reads, updates, and deletes contacts.

Simple meaning:

CRUD is the basic pattern for managing data.

5. CRUD with an Array

Before storing data in the browser, first understand CRUD with a normal array.

```
let items = [];
```

This array stores the current app data.

Create

Create means add new data.

```
function createItem(item) {  
  items.push(item);  
}
```

Example:

```
createItem("Study JavaScript");  
console.log(items);
```

Output:

```
["Study JavaScript"]
```

Simple meaning:

Create adds something new.

Read

Read means get existing data.


```
function readItems() {  
  return items;  
}
```

Example:

```
console.log(readItems());
```

Simple meaning:

Read shows or returns what already exists.

Update

Update means change existing data.

```
function updateItem(index, newItem) {  
  items[index] = newItem;  
}
```

Example:

```
items = ["Old task"];  
updateItem(0, "Updated task");  
console.log(items);
```

Output:

```
["Updated task"]
```

Simple meaning:

Update changes something that already exists.

Delete

Delete means remove existing data.

```
function deleteItem(index) {  
  items.splice(index, 1);  
}
```

Example:

```
items = ["Task A", "Task B"];  
deleteItem(0);  
console.log(items);
```

Output:

```
["Task B"]
```

Simple meaning:

Delete removes something.

6. Why Array CRUD Is Not Enough

This array version works:

```
let items = [];  
  
function createItem(item) {  
  items.push(item);  
}
```

But it has a limitation.

The data is only in memory.

If the page reloads, the array resets.

Example:

```
let items = [];
```

The browser starts from this line again.

So saved items disappear.

To fix this, you need persistence.

7. Persistence

Persistence means data remains available later.

In Sprint 17, persistence means:

Store data in the browser so it survives refresh.

Example:

```
localStorage.setItem("username", "Sangeeth");
```

Now the browser remembers the username.

Read it later:

```
const username = localStorage.getItem("username");  
console.log(username);
```

Output:

```
Sangeeth
```

Simple meaning:

Persistence lets a browser app remember simple data.

8. Browser Storage

Browser storage means storage tools built into the browser.

Sprint 17 focuses mainly on:

```
localStorage  
sessionStorage
```

It also introduces these at a beginner level:

```
cookies  
Cache API  
IndexedDB  
service workers
```

Browser storage is useful for simple app data.

Examples:

- saved notes
- todo lists
- theme preference
- draft text
- small settings
- beginner project data

Browser storage is not a replacement for a real secure server database.

9. localStorage

`localStorage` is a browser storage object.

It stores key-value pairs.

Example:

```
localStorage.setItem("theme", "dark");
```

This stores:

```
key: "theme"  
value: "dark"
```

Read it later:

```
const theme = localStorage.getItem("theme");
console.log(theme);
```

Output:

dark

Simple meaning:

`localStorage` stores data for the current website in the user's browser.

10. Key-Value Storage

Browser storage uses keys and values.

A key is the name used to find the stored value.

A value is the data stored under that key.

Example:

```
localStorage.setItem("language", "JavaScript");
```

Here:

Part	Value
Key	"language"
Value	"JavaScript"

To read the value, use the same key:

```
const language = localStorage.getItem("language");
console.log(language);
```

Output:

JavaScript

Beginner rule:

Use clear key names so your stored data is easy to understand.

Good keys:

```
"tasks"  
"notes"  
"theme"  
"draftMessage"  
"userSettings"
```

Weak keys:

```
"x"  
"data"  
"thing"
```

11. localStorage Methods

`setItem()`

`setItem()` saves a value.

```
localStorage.setItem("name", "Ada");
```

Simple meaning:

Save this value under this key.

`getItem()`

`getItem()` reads a value.

```
const name = localStorage.getItem("name");  
console.log(name);
```

Output:

Ada

Simple meaning:

Get the value stored under this key.

`removeItem()`

`removeItem()` removes one stored key.

```
localStorage.removeItem("name");
```

Simple meaning:

Delete this one stored item.

`clear()`

`clear()` removes all `localStorage` data for the current site.

```
localStorage.clear();
```

Simple meaning:

Delete all stored `localStorage` data for this website.

Beginner warning:

Be careful with `clear()` because it removes everything in `localStorage` for that site.

12. `localStorage` Stores Strings

This is one of the most important Sprint 17 rules:

`localStorage` stores strings.

This works correctly:

```
localStorage.setItem("theme", "dark");
```

The value `"dark"` is already a string.

But this causes a problem:

```
const user = { name: "Ada", score: 90 };  
  
localStorage.setItem("user", user);
```

You might expect the object to be saved.

But the browser stores something like:

```
"[object Object]"
```

That is not useful.

Why?

Because `localStorage` stores strings, and the object was converted badly.

Correct version:

```
const user = { name: "Ada", score: 90 };  
  
localStorage.setItem("user", JSON.stringify(user));
```

Simple rule:

Before storing arrays or objects, convert them to JSON strings.

13. JSON Review

JSON means JavaScript Object Notation.

JSON is a text format for representing data.

A JavaScript object looks like this:


```
const user = {  
  name: "Ada",  
  score: 90  
};
```

A JSON string can look like this:

```
'{"name": "Ada", "score": 90}'
```

Simple meaning:

JSON lets JavaScript data be stored or sent as text.

Sprint 17 uses JSON because browser storage stores strings.

14. JSON.stringify()

`JSON.stringify()` converts a JavaScript value into a JSON string.

Example:

```
const task = {  
  text: "Study Sprint 17",  
  done: false  
};  
  
const taskString = JSON.stringify(task);  
  
console.log(taskString);
```

Output:

```
{"text": "Study Sprint 17", "done": false}
```

Simple meaning:

`JSON.stringify()` changes an object or array into text.

Use it before saving arrays or objects into `localStorage`.

Example:

```
const tasks = [
  { text: "Study", done: false },
  { text: "Practice", done: true }
];

localStorage.setItem("tasks", JSON.stringify(tasks));
```

15. JSON.parse()

`JSON.parse()` converts a JSON string back into JavaScript data.

Example:

```
const taskString = '{"text":"Study Sprint 17","done":false}';

const taskObject = JSON.parse(taskString);

console.log(taskObject.text);
console.log(taskObject.done);
```

Output:

```
Study Sprint 17
false
```

Simple meaning:

`JSON.parse()` changes JSON text back into a usable JavaScript object or array.

Use it after reading arrays or objects from `localStorage`.

Example:

```
const savedTasks = localStorage.getItem("tasks");
const tasks = JSON.parse(savedTasks);
```

16. JSON.stringify() vs JSON.parse()

Method	Direction	Use Case
<code>JSON.stringify()</code>	JavaScript value → string	Before saving
<code>JSON.parse()</code>	string → JavaScript value	After loading

Memory trick:

```
JSON.stringify() // prepare data for storage
JSON.parse()    // restore data after storage
```

Another way to remember:

```
Before localStorage.setItem(): stringify
After  localStorage.getItem(): parse
```

17. Basic Save and Load Pattern

This is the basic Sprint 17 storage pattern:

```
const tasks = [
  { text: "Study", done: false }
];

localStorage.setItem("tasks", JSON.stringify(tasks));
```

This saves the array.

To load it:

```
const savedTasks = JSON.parse(localStorage.getItem("tasks"));
console.log(savedTasks);
```

Output:

```
[
  { text: "Study", done: false }
]
```

Simple pattern:

Save:
JavaScript data → JSON.stringify() → localStorage

Load:
localStorage → JSON.parse() → JavaScript data

18. What Happens If a Key Does Not Exist?

If you ask for a key that does not exist, `localStorage.getItem()` returns `null`.

Example:

```
const savedTasks = localStorage.getItem("tasks");
console.log(savedTasks);
```

If there is no `"tasks"` key, output is:

```
null
```

Simple meaning:

`null` means there is no stored value for that key.

This is why beginner code often uses a fallback value.

19. The `|| []` Fallback Pattern

This line is very common in Sprint 17:

```
let tasks = JSON.parse(localStorage.getItem("tasks")) || [];
```

Read it slowly:

```
localStorage.getItem("tasks")
```

Get the saved tasks string.

```
JSON.parse(localStorage.getItem("tasks"))
```

Convert the saved JSON string back into JavaScript data.

```
|| []
```

If there is no saved data, use an empty array.

Simple meaning:

Load saved tasks. If saved tasks do not exist, start with an empty task list.

20. Why `|| []` Works

If the key does not exist:

```
localStorage.getItem("tasks")
```

returns:

```
null
```

Then:

```
JSON.parse(null)
```

returns:

```
null
```

So this expression:

```
JSON.parse(localStorage.getItem("tasks")) || []
```

becomes:

```
null || []
```

Because `null` is falsy, JavaScript uses:

```
[]
```

Result:

```
let tasks = [];
```

Simple meaning:

The app starts with an empty array when no saved data exists.

21. The Main Saved Task List Pattern

This is one of the most important Sprint 17 examples.

```
let tasks = JSON.parse(localStorage.getItem("tasks")) || [];  
  
function saveTasks() {  
  localStorage.setItem("tasks", JSON.stringify(tasks));  
}  
  
function createTask(text) {  
  tasks.push({  
    text: text,  
    done: false  
  });  
  
  saveTasks();  
}  
  
function readTasks() {  
  return tasks;  
}
```

```
function updateTask(index, newText) {
  tasks[index].text = newText;

  saveTasks();
}

function toggleTask(index) {
  tasks[index].done = !tasks[index].done;

  saveTasks();
}

function deleteTask(index) {
  tasks.splice(index, 1);

  saveTasks();
}
```

This pattern combines:

- arrays
- objects
- functions
- JSON
- localStorage
- CRUD

22. Reading the Saved Task Pattern Step by Step

Step 1: Load existing tasks

```
let tasks = JSON.parse(localStorage.getItem("tasks")) || [];
```

This means:

Try to load saved tasks. If there are none, start with an empty array.

Step 2: Create a save function

```
function saveTasks() {  
  localStorage.setItem("tasks", JSON.stringify(tasks));  
}
```

This means:

Convert the task array into a JSON string and save it.

Step 3: Create a task

```
function createTask(text) {  
  tasks.push({  
    text: text,  
    done: false  
  });  
  
  saveTasks();  
}
```

This does two things:

1. Adds a new task object to the array.
2. Saves the updated array.

Important pattern:

```
Change data  
Save data
```

Step 4: Read tasks

```
function readTasks() {  
  return tasks;  
}
```

This returns the current array.

Simple meaning:

Give back the current task list.

Step 5: Update task text

```
function updateTask(index, newText) {  
  tasks[index].text = newText;  
  
  saveTasks();  
}
```

This changes the text of one task and then saves.

Step 6: Toggle task completion

```
function toggleTask(index) {  
  tasks[index].done = !tasks[index].done;  
  
  saveTasks();  
}
```

This flips `done`.

If `done` is `false`, it becomes `true`.

If `done` is `true`, it becomes `false`.

Simple meaning:

Mark incomplete as complete, or complete as incomplete.

Step 7: Delete a task

```
function deleteTask(index) {  
  tasks.splice(index, 1);  
  
  saveTasks();  
}
```

This removes one task from the array and then saves.

23. The Most Important Sprint 17 Habit

Every time your data changes, save it.

Create:

```
tasks.push(newTask);  
saveTasks();
```

Update:

```
tasks[index].text = newText;  
saveTasks();
```

Delete:

```
tasks.splice(index, 1);  
saveTasks();
```

Simple rule:

Change the array, then save the array.

If you forget to save, the change only exists in memory.

When the page reloads, the unsaved change disappears.

24. Safer Loading with try...catch

The simple pattern is useful:

```
let tasks = JSON.parse(localStorage.getItem("tasks")) || [];
```

But it can break if the stored data is damaged or not valid JSON.

Example of broken stored data:

```
not valid JSON
```

If JavaScript tries to parse broken JSON, it throws an error.

So a safer approach uses `try...catch`.

```
function safeLoadTasks() {
  try {
    const saved = JSON.parse(localStorage.getItem("tasks")) || [];

    if (Array.isArray(saved)) {
      return saved;
    }

    return [];
  } catch (error) {
    return [];
  }
}

let tasks = safeLoadTasks();
```

Simple meaning:

Try to load the saved tasks. If anything is wrong, use an empty array instead of crashing.

25. Why Safe Loading Matters

Stored data can be bad for several reasons:

- The app used an older data format before.
- The user changed localStorage manually in DevTools.
- The storage value became broken.
- Your code saved the wrong type by mistake.
- The key exists, but its value is not the array you expected.

Bad loading can crash the app.

Safe loading protects the app.

26. Understanding try...catch in Storage Code

`try...catch` lets your app handle errors without fully crashing.

Basic shape:

```
try {  
  // risky code  
} catch (error) {  
  // fallback code  
}
```

In Sprint 17, the risky code is usually:

```
JSON.parse(...)
```

because parsing can fail.

Example:

```
function safeLoadTasks() {  
  try {  
    return JSON.parse(localStorage.getItem("tasks")) || [];  
  } catch (error) {  
    return [];  
  }  
}
```

Simple meaning:

If loading works, return saved data. If loading fails, return an empty array.

27. Why Array.isArray() Is Used

This check appears in safer code:

```
if (Array.isArray(saved)) {  
  return saved;  
}  
  
return [];
```

Why?

Because the app expects tasks to be an array.

Valid:

```
[
  { text: "Study", done: false }
]
```

Invalid for this app:

```
{ text: "Study", done: false }
```

That is an object, not an array.

Invalid:

```
"hello"
```

That is a string, not an array.

Simple meaning:

`Array.isArray()` checks that the loaded data has the shape your app expects.

28. sessionStorage

`sessionStorage` is similar to `localStorage`, but it is more temporary.

Example:

```
sessionStorage.setItem("draft", "Temporary text");
```

Read it:

```
const draft = sessionStorage.getItem("draft");
console.log(draft);
```

Simple meaning:

`sessionStorage` stores data only for the current browser tab session.

29. localStorage vs sessionStorage

Feature	localStorage	sessionStorage
Survives page refresh?	Yes	Yes
Survives closing the tab?	Usually yes	No
Scope	Website/browser storage	Current tab session
Good for	Saved preferences, notes, todos	Temporary draft or session-only data

Use `localStorage` when:

- the data should remain later
- the app should remember after reopening
- the data is not sensitive

Use `sessionStorage` when:

- the data should be temporary
- the data should disappear when the tab closes
- the data only matters during one session

30. Removing Stored Data

Sometimes you need to remove saved data.

Remove one item

```
localStorage.removeItem("tasks");
```

This removes only the `"tasks"` key.

Simple meaning:

Delete one stored value.

Clear all storage for the site

```
localStorage.clear();
```

This removes all `localStorage` keys for the current site.

Simple meaning:

Delete everything this site saved in localStorage.

Beginner warning:

Prefer `removeItem()` when you only want to delete one app value.

31. Clear Notes Example

Suppose your app stores notes under the key `"notes"`.

You can clear only those notes:

```
function clearNotes() {  
  notes = [];  
  localStorage.removeItem("notes");  
}
```

This does two things:

1. Clears the in-memory array.
2. Removes the saved notes from `localStorage`.

Why both?

Because the app has two places to consider:

```
memory array  
browser storage
```

If you only remove from storage but keep the memory array, the current page may still show old notes.

If you only clear memory but do not remove storage, the notes may come back after refresh.

32. HTTP Method Mapping

Sprint 17 also connects CRUD to HTTP methods.

This prepares you for API work later.

CRUD Operation	HTTP Method	Simple Meaning
Create	POST	Add new data
Read	GET	Fetch existing data
Replace/update	PUT	Replace existing data
Partial update	PATCH	Change part of existing data
Delete	DELETE	Remove data

Simple examples:

```
Create a task -> POST
Read tasks   -> GET
Update a task -> PUT or PATCH
Delete a task -> DELETE
```

Beginner note:

In Sprint 17, you may not deeply use `fetch()` yet, but this mapping helps you understand how CRUD connects to server APIs.

33. CRUD: Browser Storage vs Server API

With browser storage, data stays on the user's device.

Example:

```
localStorage.setItem("tasks", JSON.stringify(tasks));
```

With a server API, data is sent to a server.

Conceptual example:

```
fetch("/tasks", {
  method: "POST",
  body: JSON.stringify(newTask)
});
```

Beginner distinction:

Storage Type	Where Data Lives	Good For
<code>localStorage</code>	User's browser	Simple practice apps, preferences
Server database	Remote server	Real accounts, shared data, secure storage

Simple meaning:

Browser storage is local. Server storage is shared and controlled by the backend.

34. Client-Side Storage Safety

Client-side storage means data stored in the user's browser.

This includes:

- `localStorage`
- `sessionStorage`
- cookies
- IndexedDB
- Cache API

Important rule:

Browser storage is useful, but it is not trusted secure storage.

Do not store sensitive secrets in browser storage.

Avoid storing:

```
passwords
secret API keys
private tokens
highly sensitive personal data
```

Why?

Because browser storage can be:

- inspected by the user
- modified by the user
- deleted by the user
- accessed by unsafe JavaScript if your app has a security problem

Simple meaning:

Do not treat data in the browser as secret or fully trustworthy.

35. Trust Boundaries

A trust boundary is a line between data you can trust and data you cannot fully trust.

In Sprint 17, the important trust boundary is:

Data from browser storage should not be blindly trusted.

Even if your own app saved the data, the data can change later.

Example:

```
const tasks = JSON.parse(localStorage.getItem("tasks"));
```

Do not assume this is always a valid task array.

Safer:

```
const tasks = safeLoadTasks();
```

Simple meaning:

Validate stored data when you load it.

36. Good Uses of localStorage

Good beginner uses:

```
theme preference  
saved notes  
todo list data  
draft text  
small app settings  
practice project data  
recent search term  
simple UI preference
```

Example:

```
localStorage.setItem("theme", "dark");
```

Example:

```
localStorage.setItem("notes", JSON.stringify(notes));
```

These are reasonable because they are not secret.

37. Bad Uses of localStorage

Bad uses:

```
localStorage.setItem("password", "myPassword123");  
localStorage.setItem("apiKey", "secret-key");  
localStorage.setItem("privateToken", "secret-token");
```

Why bad?

Because the data is stored on the client side.

Client-side data is not private enough for secrets.

Beginner rule:

Never store passwords or secret keys in `localStorage`.

38. Cookies at a Beginner Level

Cookies are small pieces of data stored by the browser.

They are often used by servers.

Example uses:

- session identification
- login state handled by the server
- small preferences

Beginner meaning:

A cookie is small browser-stored data that can be connected to HTTP requests.

Important beginner rule:

Do not use cookies as a random place to store sensitive app data.

Cookies can be controlled with special security settings, but that is more advanced.

For Sprint 17, you only need basic awareness.

39. Cache API at a Beginner Level

The Cache API stores request and response data.

Simple meaning:

The Cache API can help a web app reuse files or network responses.

Example concept:

The browser downloads a file once.
Later, the app can reuse a cached copy.

The Cache API is often used with service workers.

Beginner rule:

Cache API is not mainly for normal task-list data. It is more for cached files and network responses.

40. IndexedDB at a Beginner Level

IndexedDB is a browser database.

It can store larger and more structured data than `localStorage`.

Simple meaning:

IndexedDB is for bigger client-side storage needs.

Use IndexedDB when:

- data is too large for simple `localStorage`
- data is more complex
- you need more database-like browser storage
- you are building an offline-capable app

Beginner rule:

Start with `localStorage` for simple beginner apps. Learn IndexedDB later for larger browser databases.

41. Service Workers at a Beginner Level

A service worker is a script that can run between the browser and the network.

Simple meaning:

A service worker can intercept network requests and work with cached data.

Service workers are often used for:

- offline support
- caching files
- progressive web apps
- background-like browser behavior

Beginner rule:

You do not need to master service workers in Sprint 17. Just recognize that they can work with caching and offline behavior.

42. Storage Tool Comparison

Tool	Beginner Meaning	Common Use
<code>localStorage</code>	Simple persistent key-value storage	Notes, todos, preferences
<code>sessionStorage</code>	Temporary tab/session key-value storage	Drafts, session-only state
Cookies	Small browser data often connected to HTTP	Server-managed session data
Cache API	Stores request/response pairs	Offline files, cached assets
IndexedDB	Browser database	Larger structured data

Tool	Beginner Meaning	Common Use
Service worker	Script between browser and network	Offline support and caching

For Sprint 17 practice, focus mainly on:

```
localStorage
sessionStorage
JSON.stringify()
JSON.parse()
```

43. Mini-Build: Saved Notes List

A good Sprint 17 project is:

Build a saved notes list with add, edit, delete, and clear all.

The data can look like this:

```
{
  text: "Learn localStorage",
  createdAt: "2026-05-07T00:00:00.000Z"
}
```

Each note is an object.

The full notes list is an array:

```
const notes = [
  {
    text: "Learn localStorage",
    createdAt: "2026-05-07T00:00:00.000Z"
  },
  {
    text: "Practice JSON.parse",
    createdAt: "2026-05-07T00:10:00.000Z"
  }
];
```

44. Saved Notes Core JavaScript

```
function safeLoadNotes() {
  try {
    const saved = JSON.parse(localStorage.getItem("notes")) || [];

    return Array.isArray(saved) ? saved : [];
  } catch (error) {
    return [];
  }
}

let notes = safeLoadNotes();

function saveNotes() {
  localStorage.setItem("notes", JSON.stringify(notes));
}

function addNote(text) {
  notes.push({
    text: text,
    createdAt: new Date().toISOString()
  });

  saveNotes();
}

function editNote(index, newText) {
  notes[index].text = newText;

  saveNotes();
}

function deleteNote(index) {
  notes.splice(index, 1);

  saveNotes();
}

function clearNotes() {
  notes = [];

  localStorage.removeItem("notes");
}
```

45. Reading the Saved Notes Code Step by Step

`safeLoadNotes()`

```
function safeLoadNotes() {  
  try {  
    const saved = JSON.parse(localStorage.getItem("notes")) || [];  
  
    return Array.isArray(saved) ? saved : [];  
  } catch (error) {  
    return [];  
  }  
}
```

This tries to load notes safely.

If the saved notes are valid and are an array, it returns them.

If anything is wrong, it returns an empty array.

`let notes = safeLoadNotes();`

```
let notes = safeLoadNotes();
```

This initializes the app data.

Simple meaning:

Start the app with saved notes if they exist, or an empty list if they do not.

`saveNotes()`

```
function saveNotes() {  
  localStorage.setItem("notes", JSON.stringify(notes));  
}
```

This saves the current notes array.

It must use `JSON.stringify()` because `localStorage` stores strings.

addNote()

```
function addNote(text) {  
  notes.push({  
    text: text,  
    createdAt: new Date().toISOString()  
  });  
  
  saveNotes();  
}
```

This creates a new note object and saves the updated notes array.

`new Date().toISOString()` adds a machine-friendly creation time.

editNote()

```
function editNote(index, newText) {  
  notes[index].text = newText;  
  
  saveNotes();  
}
```

This updates one note's text and saves.

deleteNote()

```
function deleteNote(index) {  
  notes.splice(index, 1);  
  
  saveNotes();  
}
```

This removes one note and saves.

clearNotes()

```
function clearNotes() {  
  notes = [];
```

```
localStorage.removeItem("notes");  
}
```

This clears both:

- the in-memory notes array
- the saved notes in browser storage

46. How Sprint 17 Uses Earlier Sprints

Sprint 17 combines many earlier skills.

Earlier Skill	Used In Sprint 17 For
Variables	Store current app data
Strings	Storage keys and text values
Booleans	Task completion state
Functions	CRUD operations
Arrays	Lists of notes or tasks
Objects	Individual notes or tasks
JSON	Convert data to and from strings
Dates	Store creation timestamps
DOM	Display saved data on the page
Events	Save data after user actions
Forms	Collect note/task input
Accessibility	Announce saved/deleted status

Simple meaning:

Sprint 17 is not isolated. It combines many previous JavaScript skills into a practical app pattern.

47. Common Beginner Mistakes

Mistake 1: Storing Objects Directly

Wrong:

```
localStorage.setItem("user", { name: "Ada" });
```

Problem:

The object is not stored correctly.

Correct:

```
localStorage.setItem("user", JSON.stringify({ name: "Ada" }));
```

Rule:

Use `JSON.stringify()` before storing objects or arrays.

Mistake 2: Forgetting to Parse

Wrong:

```
const user = localStorage.getItem("user");  
console.log(user.name);
```

Problem:

`user` is a string, not an object.

Correct:

```
const user = JSON.parse(localStorage.getItem("user"));  
console.log(user.name);
```

Rule:

Use `JSON.parse()` after loading JSON data.

Mistake 3: Updating Memory but Not Saving

Wrong:

```
tasks.push({ text: "New task", done: false });
```

Problem:

The task is added only to memory.

If the page reloads, it may disappear.

Correct:

```
tasks.push({ text: "New task", done: false });  
saveTasks();
```

Rule:

After changing the array, save the array.

Mistake 4: Trusting Stored Data Blindly

Weak:

```
const tasks = JSON.parse(localStorage.getItem("tasks"));
```

Problem:

The stored data might be broken or the wrong type.

Safer:

```
const tasks = safeLoadTasks();
```

Rule:

Validate loaded data before using it.

Mistake 5: Using `clear()` Too Aggressively

Risky:

```
localStorage.clear();
```

Problem:

This clears all `localStorage` keys for the current site.

Safer if you only want to delete tasks:

```
localStorage.removeItem("tasks");
```

Rule:

Use `removeItem()` when deleting one stored value.

Mistake 6: Storing Sensitive Data

Wrong:

```
localStorage.setItem("password", "secret123");  
localStorage.setItem("apiKey", "my-secret-key");
```

Problem:

Browser storage is not secure secret storage.

Rule:

Do not store passwords, secret API keys, or private tokens in `localStorage`.

Mistake 7: Using One Generic Storage Key for Everything

Weak:

```
localStorage.setItem("data", JSON.stringify(tasks));
```

Problem:

The key name is too vague.

Better:

```
localStorage.setItem("tasks", JSON.stringify(tasks));
```

Rule:

Use specific storage keys.

48. Beginner Storage Checklist

When using browser storage, check these questions:

1. Is the data safe to store in the browser?
2. Am I using a clear key name?
3. If the data is an object or array, did I use `JSON.stringify()` before saving?
4. When loading it, did I use `JSON.parse()`?
5. What happens if the key does not exist?
6. Do I need a fallback like `|| []`?
7. Could the stored data be broken?
8. Should I use `try...catch`?
9. Did I save after every create, update, or delete?
10. Should I remove one item with `removeItem()` instead of clearing everything?

49. Sprint 17 Mental Model

A saved browser app usually works like this:

```
Page loads
↓
JavaScript loads saved data from localStorage
↓
If no saved data exists, use a default value
↓
User creates, updates, or deletes data
↓
JavaScript changes the in-memory array
↓
JavaScript saves the updated array to localStorage
↓
Page refreshes
```

↓

JavaScript loads the saved data again

Shorter version:

Load → Change → Save → Load again later

50. Minimum Sprint 17 Pattern to Memorize

You should understand this pattern clearly:

```
let items = JSON.parse(localStorage.getItem("items")) || [];  
  
function save() {  
  localStorage.setItem("items", JSON.stringify(items));  
}
```

Line 1:

```
let items = JSON.parse(localStorage.getItem("items")) || [];
```

Means:

Load saved items. If there are no saved items, use an empty array.

Line 2:

```
function save() {  
  localStorage.setItem("items", JSON.stringify(items));  
}
```

Means:

Convert the current items array into a string and save it.

This is the smallest core idea of Sprint 17.

51. Practice: Simple Tasks Storage

Try to predict what this code does:

```
let tasks = JSON.parse(localStorage.getItem("tasks")) || [];  
  
function saveTasks() {  
  localStorage.setItem("tasks", JSON.stringify(tasks));  
}  
  
function addTask(text) {  
  tasks.push({  
    text: text,  
    done: false  
  });  
  
  saveTasks();  
}  
  
addTask("Learn CRUD");
```

Expected result:

- The task array gets a new task.
- The task is saved into `localStorage`.
- If the page refreshes, the task can be loaded again.

52. Practice: Read from Storage

Code:

```
const saved = localStorage.getItem("tasks");  
console.log(saved);
```

What type is `saved`?

Answer:

```
string or null
```

It is a string if the key exists.

It is `null` if the key does not exist.

53. Practice: Parse Saved Data

Code:

```
const tasks = JSON.parse(localStorage.getItem("tasks")) || [];  
console.log(tasks);
```

What type should `tasks` be?

Answer:

array

If saved tasks exist, it should become the saved array.

If no saved tasks exist, it becomes an empty array.

54. Practice: Add Created Date

Code:

```
function addNote(text) {  
  const note = {  
    text: text,  
    createdAt: new Date().toISOString()  
  };  
  
  notes.push(note);  
  saveNotes();  
}
```

What does `createdAt` do?

Answer:

It stores the date and time when the note was created.

Why use `toISOString()`?

It creates a standard machine-friendly date string.

55. Practice: Delete One Item from Storage

If you want to remove only saved notes, use:

```
localStorage.removeItem("notes");
```

If you use this:

```
localStorage.clear();
```

You remove all localStorage values for the current site.

Beginner rule:

Use the smallest delete operation that matches your goal.

56. Sprint 17 Vocabulary

CRUD

Create, read, update, and delete data.

Persistence

Data remains available after refresh or later browser use.

localStorage

Browser storage that keeps string data for a site.

sessionStorage

Browser storage that keeps string data for the current tab session.

Key

The name used to store and retrieve a value.

Value

The data stored under a key.

JSON.stringify()

Converts JavaScript data into a JSON string.

JSON.parse()

Converts a JSON string back into JavaScript data.

Fallback value

A default value used when real data does not exist.

Example:

```
const items = savedItems || [];
```

try...catch

A JavaScript structure for handling errors safely.

Trust boundary

A line between trusted and untrusted data.

In Sprint 17, browser-stored data should not be blindly trusted.

57. Sprint 17 Checkpoint Questions

You understand Sprint 17 when you can answer these:

1. What does CRUD stand for?
2. What does persistence mean?
3. Why does normal array data disappear after refresh?
4. What is `localStorage`?
5. What is `sessionStorage`?
6. What is the difference between `localStorage` and `sessionStorage`?
7. What does `localStorage.setItem()` do?
8. What does `localStorage.getItem()` do?
9. What does `localStorage.getItem()` return if the key does not exist?
10. Why does `localStorage` need strings?

11. Why do you need `JSON.stringify()` before storing an object or array?
 12. Why do you need `JSON.parse()` after reading stored JSON?
 13. Why is `|| []` useful when loading saved arrays?
 14. Why can `JSON.parse()` be risky?
 15. Why is `try...catch` useful when loading saved data?
 16. Why should you check `Array.isArray(saved)`?
 17. What should happen after every create, update, or delete operation?
 18. What is the difference between `removeItem()` and `clear()`?
 19. Which HTTP method maps to create?
 20. Which HTTP method maps to read?
 21. Which HTTP method maps to update?
 22. Which HTTP method maps to delete?
 23. Why should you not store passwords in `localStorage`?
 24. Why is browser storage not fully trusted?
 25. What is IndexedDB used for at a beginner level?
 26. What is the Cache API used for at a beginner level?
 27. What is a service worker at a beginner level?
-

58. Sprint 17 Final Summary

Sprint 17 teaches you how browser apps remember simple data.

The main idea is:

```
Use arrays and objects for app data.  
Use JSON.stringify() to save that data as a string.  
Use localStorage to persist it.  
Use JSON.parse() to load it again.  
Use safe loading so bad stored data does not crash your app.
```

The most important code pattern is:

```
let items = JSON.parse(localStorage.getItem("items")) || [];  
  
function saveItems() {  
  localStorage.setItem("items", JSON.stringify(items));  
}
```

The most important habit is:

```
After every create, update, or delete operation, save the updated data.
```

The most important safety rule is:

Do not store secrets in browser storage, and do not blindly trust data loaded from browser storage.

If you understand these three ideas, you understand the core of Sprint 17.